# Laravel 12 Setup & Troubleshooting

STUDENT LABORATORY GUIDE | Windows 11 + VS Code + PHP/XAMPP + Composer

Beginner-friendly instructions based on the classroom troubleshooting process. Follow the steps in order.

Example project: LaravelStudentLab.

## Step 1 — Identify the problem

```
php artisan serve
```

If PowerShell reports that **php is not recognized**, Windows cannot find the PHP command. Set up PHP before continuing with Composer or Laravel.

## Step 2 — Check the PHP executable

```
Test-Path C:\xampp\php\php.exe
```

**True:** The php.exe file exists in the expected XAMPP folder. Continue to Step 3A.
**False:** The file was not found in this specific folder. This does not necessarily mean PHP is missing from the computer. Follow Step 3B.

## Step 3A — If TRUE: Check the PHP version

```
C:\xampp\php\php.exe -v
```

Expected: PHP version details. Laravel 12 requires PHP 8.2 or later. In our example, PHP 8.2.12 worked. Continue to Step 4.

## Step 3B — If FALSE: Find or install PHP

**3B.1** Check whether Windows recognizes another PHP installation:

```
where.exe php
```

If a path appears, note its location and test **php -v**. If no path appears, PHP may still be installed but not included in PATH.

**3B.2** Open File Explorer and search existing PHP or XAMPP installation folders for **php.exe**. Avoid reinstalling XAMPP if you already have a working installation.

**3B.3** If PHP is not installed, download XAMPP from **https://www.apachefriends.org/**. Choose a release with PHP 8.2 or later, install it if permitted, and note the installation folder. On school-managed computers, ask IT for permission when necessary.

**3B.4** Check the PHP executable after installation. For the default XAMPP location, run:

```
Test-Path C:\xampp\php\php.exe
C:\xampp\php\php.exe -v
```

Expected: **True** and PHP version information. If PHP is installed elsewhere, replace the example folder with the actual path. Continue to Step 4.

## Step 4 — Make the PHP command work

For a temporary fix in the current VS Code PowerShell terminal (default XAMPP location):

```
$env:Path = "C:\xampp\php;" + $env:Path
php -v
```

If PHP is installed elsewhere, use its actual folder. For a permanent fix, open Windows Start, search for **Environment Variables**, then go to **Environment Variables > User variables > Path > Edit > New**. Add your PHP folder (for example, **C:\xampp\php**). Save, completely restart VS Code, and run **php -v** again.

## Step 5 — Check whether Composer works

```
composer --version
```

If a Composer version appears, continue to Step 8. If the command is not recognized, check a common Composer installation location:

```
Test-Path "C:\ProgramData\ComposerSetup\bin\composer.bat"
```

A **False** result means Composer was not found in that location; it may be installed elsewhere. If Composer is unavailable, download it in Step 6.

## Step 6 — Download Composer

Visit **https://getcomposer.org/download/**. Under **Windows Installer**, download and open **Composer-Setup.exe**.

## Step 7 — Install Composer

Select **Install for me only** if you do not have administrator privileges. Leave **Developer mode** unchecked. On **Settings Check**, select the correct **php.exe** (in our example, **C:\xampp\php\php.exe**). Leave proxy disabled unless your network requires it. Review the settings, click **Install**, and finish setup. Close VS Code completely and reopen it to refresh PATH.

## Step 8 — Verify PHP and Composer

```
php -v
composer --version
```

Both commands should display version information. Our example used PHP 8.2.12 and Composer 2.11.0.

## Step 9 — Create a Laravel 12 project

In the VS Code terminal, navigate to a parent folder where you can create files. Run:

```
composer create-project laravel/laravel LaravelStudentLab "^12.0"
```

Wait for dependencies and setup to finish. Database migrations may run automatically. If an error appears, read it before retrying to avoid creating duplicate project folders.

## Step 10 — Enter the correct project folder

```
cd LaravelStudentLab
```

The terminal path should now end in \LaravelStudentLab>. If you get a path-not-found error, check the folder spelling and list available folders:

```
Get-ChildItem -Directory
```

## Step 11 — Open the Laravel project in VS Code

Choose File > Open Folder and select LaravelStudentLab, or run code . if that command is available. Confirm that the project contains **artisan**, **app**, **routes**, **resources**, and **vendor**.

## Step 12 — Run Laravel

Open **Terminal > New Terminal** while the project folder is open. Run:

```
php artisan serve
```

Expected: **Server running on http://127.0.0.1:8000** (or another port if 8000 is already in use). Keep the terminal open and visit the displayed URL in your browser.

## Step 13 — Quick troubleshooting reference

**php not recognized:** Locate php.exe and fix PATH (Steps 2–4).

**composer not recognized:** Install Composer or correct PATH (Steps 5–8).

**cd: cannot find path:** Check the project folder spelling and location (Step 10).

**Could not open input file: artisan:** Enter the Laravel project directory first.

**Laravel 12 PHP version issue:** Confirm PHP 8.2 or later with php -v.

**Project installation fails:** Read the terminal error. Check PHP extensions, internet connectivity, permissions, and Composer diagnostics before retrying.

**Reminder:** The PHP folder may differ across computers. Always use the actual location rather than assuming C:\xampp\php.

# LARAVEL STUDENT LAB GUIDE

From Composer project creation to database setup and displaying records

Scope: Module 6 covers Laravel setup, MVC basics, migrations, models, seeders, and database records displayed in a Blade view. Module 7 continues the same project with forms, validation, old input, and feedback.

Environment: Windows + VS Code + PowerShell + PHP 8.2 or newer compatible version + Composer + Laravel 12. SQLite is used so students do not need to configure MySQL for this exercise.

Target output: A page at http://127.0.0.1:8000/tasks showing three records: Buy groceries, Submit report, and Fix login bug.

## 1 | Check your tools

Open VS Code > Terminal > New Terminal. Run these commands one at a time:

```
php -v
composer --version
```

Both commands must display version information. If either command is not recognized, ask your instructor to check the installation and PATH for that tool before continuing.

NOTE: You do NOT need to run composer global require laravel/installer, laravel new, or composer global config bin-dir --absolute for this method.

## 2 | Create a new Laravel project using Composer

Choose a writable project location. The example below uses Documents. If your instructor requires XAMPP htdocs, you may use C:\xampp\htdocs instead.

```
cd $HOME\Documents
composer create-project laravel/laravel LaravelStudentLab "^12.0"
```

Wait until Composer finishes and the PowerShell prompt returns. This command creates a new LaravelStudentLab folder. Do not run it again with the same folder name.

```
cd LaravelStudentLab
php artisan --version
```

Expected: a Laravel Framework 12.x version. If Composer reports a missing PHP extension or dependency problem, read the specific error before changing anything.

## 3 | Configure the SQLite database

In VS Code Explorer, open the project root .env file. Find or set:

```
DB_CONNECTION=sqlite
```

If .env contains DB_HOST, DB_PORT, DB_DATABASE, DB_USERNAME, or DB_PASSWORD settings for MySQL, they are not needed for this SQLite exercise; you may leave them commented out. Save the file.

Check whether the SQLite file already exists:

```
Test-Path .\database\database.sqlite
```

If the result is False, create the file:

```
New-Item -Path .\database\database.sqlite -ItemType File
```

If the result is True, do not create it again. Ensure the pdo_sqlite and sqlite3 PHP extensions are enabled if a driver error appears.

## 4 | Create the tasks table with a migration

In the project root terminal:

```
php artisan make:migration create_tasks_table --create=tasks
```

Open the newly created file inside database/migrations (its filename begins with a timestamp and ends with create_tasks_table.php). In the up() method, use:

```
public function up(): void
{
    Schema::create('tasks', function (Blueprint $table) {
        $table->id();
        $table->string('title');
        $table->string('status')->default('pending');
        $table->timestamps();
    });
}
```

Keep the generated down() method, which drops the tasks table. Save the file, then run:

```
php artisan migrate
```

Expected: migration completed without errors. This creates the tasks table in SQLite.

## 5 | Create the Eloquent Task model

```
php artisan make:model Task
```

Open app/Models/Task.php and replace its contents with:

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Task extends Model
{
    protected $fillable = ['title', 'status'];
}
```

Explanation: The Task model represents the tasks table. The $fillable property permits mass assignment of the title and status fields, including when using Task::create().

## 6 | Insert sample records with a seeder

```
php artisan make:seeder TaskSeeder
```

Open database/seeders/TaskSeeder.php and replace its contents with:

```
<?php

namespace Database\Seeders;

use Illuminate\Database\Seeder;
```

```
use App\Models\Task;

class TaskSeeder extends Seeder
{
    public function run(): void
    {
        Task::create(['title' => 'Buy groceries',
                      'status' => 'pending']);
        Task::create(['title' => 'Submit report',
                      'status' => 'pending']);
        Task::create(['title' => 'Fix login bug',
                      'status' => 'done']);
    }
}
```

Save, then execute:

```
php artisan db:seed --class=TaskSeeder
```

NOTE: Run this seeder only once for this exercise. Running it again inserts another set of three records, so duplicates may appear. The import use App\Models\Task; is required.

## 7 | Create a controller to retrieve the records

```
php artisan make:controller TaskController
```

Open app/Http/Controllers/TaskController.php and replace its contents with:

```
<?php

namespace App\Http\Controllers;

use App\Models\Task;
use Illuminate\View\View;

class TaskController extends Controller
{
    public function index(): View
    {
        $tasks = Task::all();

        return view('tasks.index', ['tasks' => $tasks]);
    }
}
```

Explanation: Task::all() reads all task records. view('tasks.index', ...) passes the records to the Blade page.

## 8 | Register the /tasks route

Open routes/web.php. Keep any existing welcome route. Add the controller import and the route:

```
use App\Http\Controllers\TaskController;

Route::get('/tasks', [TaskController::class, 'index']);
```

NOTE: Put the use statement with the other imports near the top of the file, and the Route::get statement with the routes. Do not paste either inside another route callback.

## 9 | Create the Blade page that displays records

Inside resources/views, create a folder named tasks. Inside it, create index.blade.php. Paste:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>My Tasks</title>
    <style>
        body { font-family: Arial, sans-serif;
               max-width: 850px; margin: 40px auto; }
        table { width: 100%; border-collapse: collapse; }
        th, td { padding: 10px; border: 1px solid #ccc;
                 text-align: left; }
        th { background: #e9f1f2; }
    </style>
</head>
<body>
    <h1>My Tasks</h1>
    <table>
        <thead>
            <tr><th>ID</th><th>Task Title</th><th>Status</th></tr>
        </thead>
        <tbody>
            @forelse ($tasks as $task)
                <tr>
                    <td>{{ $task->id }}</td>
                    <td>{{ $task->title }}</td>
                    <td>{{ ucfirst($task->status) }}</td>
                </tr>
            @empty
                <tr><td colspan="3">No tasks found.</td></tr>
            @endforelse
        </tbody>
    </table>
</body>
</html>
```

Blade uses @forelse to loop through the records, and {{ ... }} to display values safely.

## 10 | Run the application and check the output

From the LaravelStudentLab project directory, run:

```
php artisan serve
```

Open http://127.0.0.1:8000/tasks in your browser (or use the URL shown by Artisan if port 8000 is unavailable).

### Expected output

| ID | Task Title | Status |
|---|---|---|
| 1 | Buy groceries | Pending |
| 2 | Submit report | Pending |
| 3 | Fix login bug | Done |

IDs are usually 1, 2, and 3 in a fresh database; actual IDs may differ if records were inserted previously.

# 11 | Quick troubleshooting reference

### laravel is not recognized

This guide does not use laravel new. Use composer create-project instead. It does not repair the laravel command itself.

### composer is not recognized

Check Composer installation and the PATH used by the current terminal. Reopen VS Code after PATH changes.

### php is not recognized

Check PHP installation and PHP PATH, then verify php -v.

### Could not find driver (SQLite)

Enable the appropriate PHP SQLite extensions in the php.ini used by CLI PHP; restart terminal and retry.

### Target class [TaskController] does not exist

Ensure routes/web.php imports App\Http\Controllers\TaskController and the class/file name matches.

### Class "Database\Seeders\Task" not found

Add use App\Models\Task; at the top of TaskSeeder.php.

### MassAssignmentException

Check Task.php contains protected $fillable = ['title', 'status'];

### No tasks found

Check DB_CONNECTION=sqlite, migration ran, and TaskSeeder ran. Use php artisan tinker, then App\Models\Task::count();

### Failed to download / Permission denied

Wait for Composer to finish; inspect the last error. Check network, writable folder, security restrictions, and free disk space. Do not assume PATH is the issue.

### Duplicate rows

TaskSeeder was probably executed more than once. Avoid rerunning it; ask your instructor before resetting the database.

# 12 | Student submission checklist

Submit screenshots showing: (1) successful Composer project installation or php artisan --version, (2) successful migration, (3) successful seeding, (4) the /tasks page displaying three records. Include the project folder name and the code files: Task.php, TaskSeeder.php, TaskController.php, routes/web.php, and tasks/index.blade.php.

Reflection questions: What is the role of the migration? What is the difference between the model and controller? How does the Blade view receive task records? Why do we use a seeder?

End of guide.

# MODULE 7 | Forms and Request Validation

Student Laboratory Guide - Continuation of Module 6 (Laravel Data Layer)

Important: This activity is an addition to your existing Module 6 Laravel Task Manager. Do not create another Laravel project, database, migration, or seeder. Keep your existing records and working task list.

## Learning outcomes

After completing this activity, you should be able to open a Create Task form, submit data using POST and CSRF protection, validate a task title, retain previous input after errors, show feedback, and save a valid record into the same tasks table from Module 6.

## Before you start - check your Module 6 project

Open the same project folder used in Module 6 in VS Code. Confirm these already exist: app/Models/Task.php, app/Http/Controllers/TaskController.php, routes/web.php, resources/views/tasks/index.blade.php, and the SQLite database with the tasks table. Your /tasks page should already display records.

Do not replace existing working code blindly. The examples below show the required methods and additions. Keep any unrelated routes, methods, and existing layout.

## Step 1 - Add the routes

Open routes/web.php. Make sure the TaskController import is present. Add names to existing routes or add the missing routes shown below. Do not duplicate the same routes.

```
use App\Http\Controllers\TaskController;

// Module 6: existing task list (add its route name if missing)
Route::get('/tasks', [TaskController::class, 'index'])
    ->name('tasks.index');

// Module 7: open the form
Route::get('/tasks/create', [TaskController::class, 'create'])
    ->name('tasks.create');

// Module 7: receive the form submission
Route::post('/tasks', [TaskController::class, 'store'])
    ->name('tasks.store');
```

Explanation: GET /tasks displays existing records. GET /tasks/create opens the form. POST /tasks receives the submitted task. Named routes let Blade refer to routes by name instead of typing URLs.

## Step 2 - Add methods to the existing controller

Open app/Http/Controllers/TaskController.php. Keep your Module 6 index() method. Ensure these imports appear once at the top of the file, below the namespace:

```
use App\Models\Task;
use Illuminate\Http\Request;
```

Inside the existing TaskController class, add create() and store() if they are not already present:

```
public function create()
{
    return view('tasks.create');
}
```

```
public function store(Request $request)
{
    // Check the submitted Task Title
    $validated = $request->validate([
        'title' => ['required', 'string', 'max:100'],
    ]);

    // Insert into the SAME Module 6 tasks table
    Task::create([
        'title' => $validated['title'],
        'status' => 'pending',
    ]);

    // Return to task list with a success message
    return redirect()
        ->route('tasks.index')
        ->with('success', 'Task created successfully!');
}
```

Explanation: create() displays the form. store(Request $request) receives the submitted data. required rejects blank input; string expects text; max:100 limits the title to 100 characters. Laravel redirects back with validation errors and previous input when validation fails. Task::create() saves only after validation passes. New tasks receive pending status automatically.

## Step 3 - Check the existing Task model

Open app/Models/Task.php. Confirm its $fillable allows title and status. Add these fields to the existing list if necessary; do not remove unrelated model code.

```
protected $fillable = ['title', 'status'];
```

Why? Eloquent needs these fields to be mass-assignable when using Task::create().

## Step 4 - Create the new Blade form

Inside the existing resources/views/tasks folder, create a new file named create.blade.php. Paste the following complete form. This is the only new view required for this activity.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Create Task</title>
</head>
<body>
    <h2>Create New Task</h2>

    <form action="{{ route('tasks.store') }}" method="POST">
        @csrf

        <label for="title">Task Title:</label>
        <input type="text" id="title" name="title"
               value="{{ old('title') }}"
               placeholder="Enter task title">

        @error('title')
            <p style="color: red;">{{ $message }}</p>
        @enderror

        <br>
        <button type="submit">Save Task</button>
    </form>

    <p><a href="{{ route('tasks.index') }}">
        Back to Task List
    </a></p>
</body>
</html>
```

Code explanation: method="POST" submits the data; @csrf adds Laravel CSRF protection; name="title" matches the validation field; old('title') restores the typed value after failed validation; @error('title') shows the corresponding error message. Keep the textbox without HTML required or maxlength attributes during testing so Laravel server-side errors can be observed.

## Step 5 - Add a link and success feedback to your Module 6 task list

Open your existing resources/views/tasks/index.blade.php. Do not delete the Module 6 table, loop, or View Details column. Add these snippets above your existing table:

```
<a href="{{ route('tasks.create') }}">+ Add New Task</a>

@if (session('success'))
    <p style="color: green;">{{ session('success') }}</p>
@endif
```

The Add New Task link opens the new form. After a successful save, the green success message appears on the original task list. Your old records should still be visible, together with the new record.

## Step 6 - Run and test

Open the VS Code terminal in the same Module 6 project folder and run:

```
php artisan serve
```

Open http://127.0.0.1:8000/tasks to see the existing list. Click + Add New Task, or open http://127.0.0.1:8000/tasks/create directly.

## Required student test cases

| Test | Action | Expected feedback |
|---|---|---|
| A. Empty | Submit without typing a title | Red: The title field is required. |
| B. Too long | Submit a title over 100 characters | Red max-length error; old input remains |
| C. Valid | Type: Complete Laravel Activity | Redirect to /tasks; green success message; new row |

## Troubleshooting

Route [tasks.index] not defined: Ensure GET /tasks has ->name('tasks.index') in routes/web.php. Check registered routes:

```
php artisan route:list --name=tasks
```

Route or view changes not appearing: Save all files, then clear Laravel caches:

```
php artisan optimize:clear
```

View [tasks.create] not found: Verify the exact file path: resources/views/tasks/create.blade.php.

Mass assignment error: Confirm title and status are included in the Task model $fillable property.

Database error when saving: Confirm you are running the original Module 6 project, that the tasks table exists, and that its title and status columns match the code.

419 Page Expired: Check that @csrf is inside the POST form and that sessions are configured and working.

Old input or errors not appearing: Ensure you use a normal web route and the same form field name title in the input, validation rules, old(), and @error().

## Submission checklist

Before submitting, verify: (1) original Module 6 task records still display; (2) Add New Task opens the form; (3) empty title shows error; (4) overlong title shows error and retains previous text; (5) valid title saves as pending; (6) success message appears on the task list.

Suggested evidence: Screenshots of the Create Task form, a validation error with retained input, and the task list showing the newly added record and success message.

## Scope reminder

This activity covers Module 7 pages 1-15 only: forms, POST requests, CSRF, Request validation, old input, error feedback, saving, and success feedback. No new project, migration, seeder, file upload, edit, update, or delete feature is required.

# Additional Troubleshooting & Continuity Checks

Use these checks only when an error appears. Run commands in the VS Code PowerShell terminal from the **LaravelStudentLab** project folder unless otherwise specified.

## 1 | Check the PHP version and required extensions

```
php -v
php -m
```

Laravel 12 requires PHP 8.2 or newer. For SQLite, confirm that **pdo_sqlite** and **sqlite3** appear in the module list. If they are missing, enable the extensions in the php.ini used by your command-line PHP, then reopen the terminal.

## 2 | Confirm the active database and migrations

```
php artisan migrate:status
Test-Path .\database\database.sqlite
```

The tasks migration should show as completed, and the SQLite file should exist. If migrations have not run, check **DB_CONNECTION=sqlite** in .env, then run **php artisan migrate**. Do not recreate the project or database if they already exist.

## 3 | Verify named routes before Module 7

```
php artisan route:list --name=tasks
```

Module 7 requires **tasks.index** (GET /tasks), **tasks.create** (GET /tasks/create), and **tasks.store** (POST /tasks). Add names to existing routes instead of creating duplicate routes. A missing tasks.index name causes Route [tasks.index] not defined.

## 4 | Refresh cached routes and views

```
php artisan optimize:clear
```

Save your edited files first. Then refresh the browser. Do not run database reset commands as a general troubleshooting step; they may delete your existing Module 6 records.

## 5 | Confirm the task model and avoid duplicate sample records

```
php artisan tinker
\App\Models\Task::count();
exit
```

Check that **app/Models/Task.php** contains the title and status fields in $fillable. If three sample tasks are already visible, do not rerun TaskSeeder; doing so inserts duplicates.

## 6 | Continue Module 7 in the same project

Keep your existing migration, seeder, task list, and records. Update the existing TaskController and routes/web.php, create **resources/views/tasks/create.blade.php**, and add the Create Task link and success message to the existing index view. Test blank input, a title longer than 100 characters, and a valid title. The form should retain old input and show validation feedback.